

GITHUB COPILOT CLI

Ronald A. Sumida

ronaldasumida@gmail.com

Agenda

- What is the GitHub Copilot CLI?
- Why the GitHub Copilot CLI?
- Installation
 - Windows terminal
 - VS Code
- Interfaces
 - Interactive
 - Programmatic
- Agentic code review
- Delegation
- Scheduling prompts
- Connecting Windows Terminal to VS Code

What is GitHub Copilot CLI?

- Brings GitHub Copilot to the CLI
 - Provides AI functionality at the command line
 - Enables use of natural language in the terminal
 - GitHub Copilot executes the corresponding commands
- AI Capabilities
 - Generative
 - Use as a chatbot
 - Agentic
 - Use autonomously
- Available with all Copilot plans
 - Must be enabled in the organization's settings

Why the GitHub Copilot CLI?

- Reduces context switching
 - Stay in the flow
 - Keep working in the terminal
- Can run headless
 - No IDE required
- Can run anywhere
 - Windows Terminal
 - PowerShell
 - macOS/Linux shells
 - SSH sessions
 - Codespaces
 - Dev Containers
- Brings AI to terminal-native work
 - Generate commands
 - Explain commands
 - Accelerate operational tasks
- Automates repetitive tasks
 - File searches
 - Boilerplate commands

Getting started with GitHub Copilot CLI

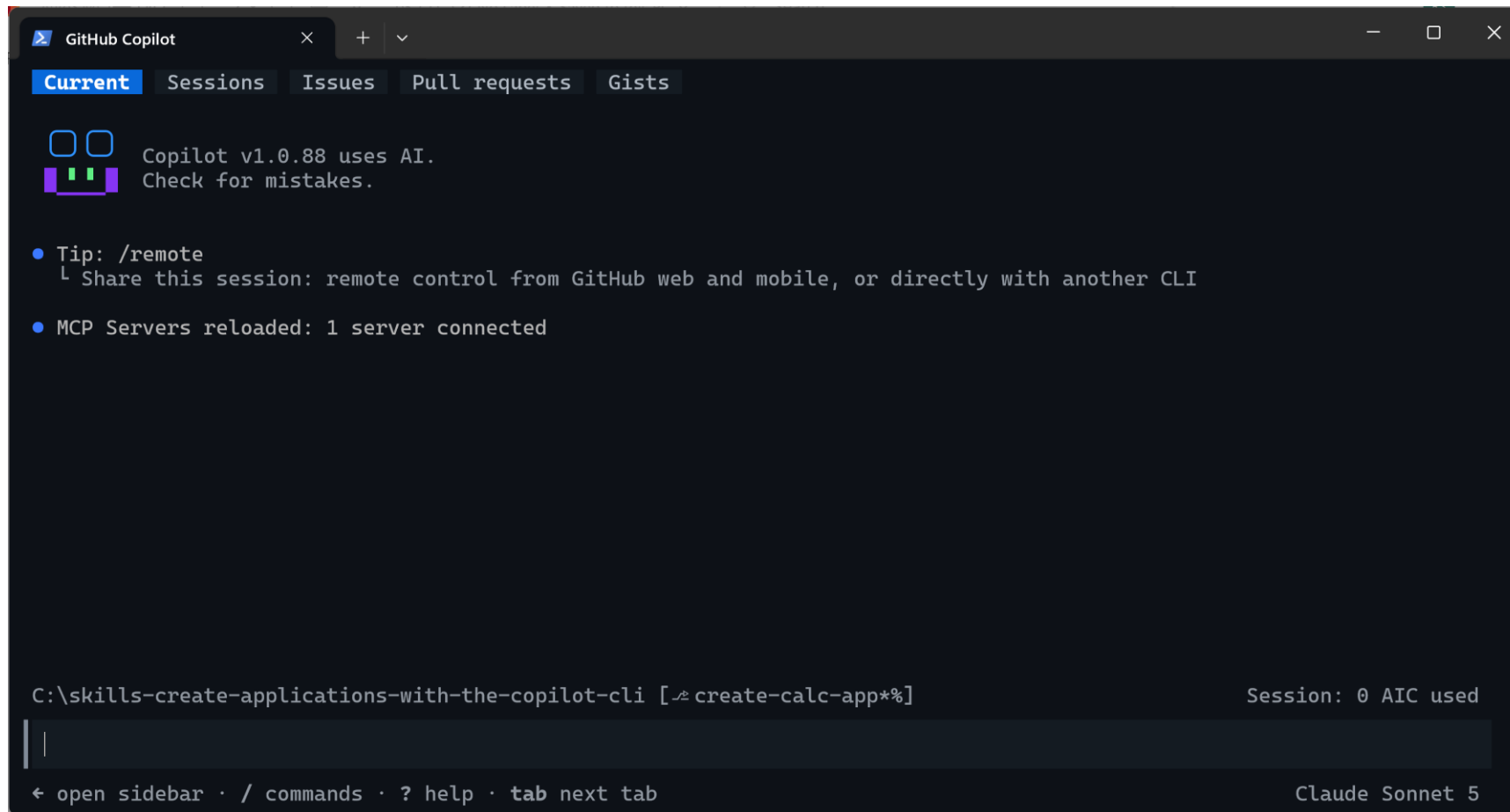
- Install with npm
 - *npm install -g @github/copilot*
- Launch the Copilot CLI
 - *copilot*
- Login
 - */login*
 - What account do you want to log into?
 - GitHub.com
 - How do you want to sign in?
 - Sign in with your browser (recommended)

Using the GitHub Copilot CLI

- Use cases
 - Local tasks
 - GitHub.com tasks
 - <https://docs.github.com/en/copilot/concepts/agents/copilot-cli/about-copilot-cli#use-cases-for-github-copilot-cli>
- Interfaces
 - Interactive
 - Copilot Chat at the command line
 - Programmatic
 - Copilot Chat for a single command

Interactive interface

- Displays five tabs and the prompt box



Interactive interface tabs

- Current
 - Active conversation
- Sessions
 - Previous conversation history
 - Press <- to display sessions sidebar
- Issues
 - GitHub issues in the current repo
 - Only shown inside a GIT repository
- Pull requests
 - GitHub Pull Requests in the current repo
 - Only shown inside a GIT repository
- Gists
 - GitHub Gists (shared code snippets)

Interactive interface modes

- Standard mode
 - Default
 - Explicitly ask for permission before making changes or running commands
- Plan mode
 - Builds a structured plan before making any changes
- Autopilot mode
 - Executes autonomously until completion without asking for permission
- Use *Shift-tab* to cycle through the modes

Interactive interface example

- GitHub.com
 - Create a private GitHub repo named "calculator-demo"
 - Clone the repo
- Local
 - Use calculator-demo as the working path
 - Create a Node.js CLI calculator app in an src folder based only on the four basic math operations
 - Run and test the calculator functions with some example operations
 - Create comprehensive unit tests for all calculator functions. Use a popular Node.js testing framework if one isn't installed
 - Run all tests and make sure they pass
- Local git
 - Add all calculator and test files to git. Commit with message "Implement basic calculator operations and tests: addition, subtraction, multiplication, division"
- GitHub.com
 - Push the changes

Programmatic interface

- Pass a single prompt to Copilot directly via the CLI
 - CLI exits after completion
- Use when an ongoing chat is not necessary
 - Copilot executes in non-interactive mode
- *copilot -p "your prompt"*
 - Can also use *--prompt*
- Needs to operate without manual approval
 - Use command line flags to allow approval

Programmatic interface approval flags

- Allow tools
 - *--allow-tool=TOOL*
 - Grants permission for a specific tool
 - *--allow-all-tools*
 - Allows every tool to run without individual approval
- Allow access to urls
 - *--allow-url-URL*
 - Allows access to a specific url or domain
 - *--allow-all-urls*
 - Allows access to all urls without approval
- Allow paths
 - *--allow-all-paths*
 - Disables file path verification
- Allow everything
 - *--allow-all*
 - *--yolo*
 - Equivalent
 - *--allow-all-tools, --allow-all-paths, --allow-all-urls*

Programmatic interface – Simple example

- *copilot -p "Explain what calculator.js does in one paragraph"*

```
Windows PowerShell
PS C:\skills-create-applications-with-the-copilot-cli> copilot -p "Explain what calculator.js does in one paragraph"
• Read copilot-cli-ref.instructions.md
  | .github\instructions\copilot-cli-ref.instructions.md
  | 91 lines read

/ Search (glob)
  | "**/calculator.js"
  | 1 file found

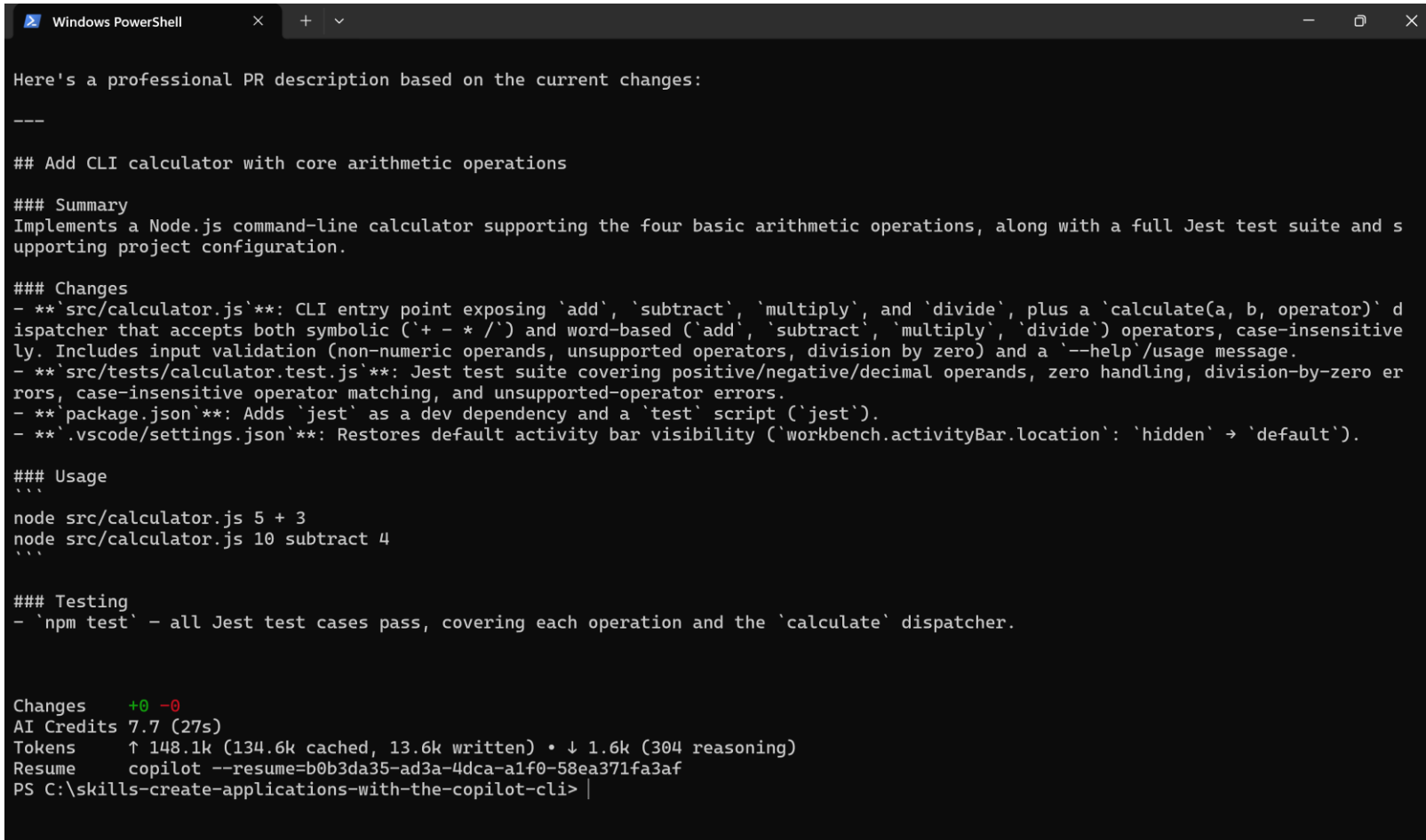
• Read calculator.js
  | src\calculator.js
  | 135 lines read

`calculator.js` is a simple Node.js command-line calculator invoked as `node calculator.js <number1> <operator> <number2>`. It supports addition, subtraction, multiplication, and division via both symbol and word operators (e.g., `+`/`add`, `-`/`subtract`, `*`/`multiply`, `/`/`divide`), mapped through an `OPERATORS` lookup table to corresponding functions in `OPERATIONS`. The `calculate` function validates the operator and dispatches to the right operation, throwing an error for division by zero or unsupported operators. `main()` parses `process.argv`, validates numeric input, prints usage/help text when arguments are missing or invalid, and prints the result or an error message; the module also exports its functions (`add`, `subtract`, `multiply`, `divide`, `calculate`) for reuse when imported elsewhere.

Changes      +0 -0
AI Credits 4.25 (12s)
Tokens       ↑ 69.6k (59.5k cached, 10.1k written) • ↓ 525 (20 reasoning)
Resume       copilot --resume=c68a46e7-90b9-4759-ac75-42b5510af272
PS C:\skills-create-applications-with-the-copilot-cli> |
```

Programmatic interface – Combining tools with AI

- *git diff | copilot -p "Write a professional PR description"*



```
Windows PowerShell

Here's a professional PR description based on the current changes:

---

## Add CLI calculator with core arithmetic operations

### Summary
Implements a Node.js command-line calculator supporting the four basic arithmetic operations, along with a full Jest test suite and supporting project configuration.

### Changes
- src/calculator.js: CLI entry point exposing add, subtract, multiply, and divide, plus a calculate(a, b, operator) dispatcher that accepts both symbolic (+ - * /) and word-based (add, subtract, multiply, divide) operators, case-insensitive. Includes input validation (non-numeric operands, unsupported operators, division by zero) and a --help usage message.
- src/tests/calculator.test.js: Jest test suite covering positive/negative/decimal operands, zero handling, division-by-zero errors, case-insensitive operator matching, and unsupported-operator errors.
- package.json: Adds jest as a dev dependency and a test script (jest).
- .vscode/settings.json: Restores default activity bar visibility (workbench.activityBar.location: hidden → default).

### Usage
```
node src/calculator.js 5 + 3
node src/calculator.js 10 subtract 4
```

### Testing
- npm test - all Jest test cases pass, covering each operation and the calculate dispatcher.

Changes      +0 -0
AI Credits 7.7 (27s)
Tokens      ↑ 148.1k (134.6k cached, 13.6k written) • ↓ 1.6k (304 reasoning)
Resume      copilot --resume=b0b3da35-ad3a-4dca-alf0-58ea371fa3af
PS C:\skills-create-applications-with-the-copilot-cli>
```

Agentic code review

- */review* command
 - Analyses code changes
 - Can run to assess changes before committing
- Result
 - Provides feedback
 - Provides suggested actions if appropriate
 - Fix
 - Commit summary of findings
 - Do nothing
 - User suggested action

Agentic code review example

Delete Last Git Commit - GitHub

Current

Sessions

Issues

Pull requests

Gists

```
cd C:\calclator\test\calculator-demo; node src/index.js __proto__ 2 3; echo EXIT=$LASTEXITCODE
```

Code Review Findings

#	Severity	File	Lines	Vulnerability	Confidence
1	MEDIUM	src\index.js	40-46	CLI silently coerces empty/whitespace numeric args (e.g. " ") to 0 via Number(), instead of rejecting as invalid input	8/10

Verified: `node src/index.js add " " 5` prints `5` (treats blank arg as `0`) rather than erroring. No other high-confidence bugs found – divide-by-zero check and core arithmetic/tests look correct.

X

 Asked user Waiting for response `"requestedSchema": Expected object, received string`

v

 Thought for 1s
The error says requestedSchema expected an object but got a string, so I must have malformed the JSON structure somewhere—likely mismatched braces. I need to redo it carefully.

o

 Asking user Waiting for response

7m 47s

Copilot needs information.

Found 1 medium-severity issue (CLI silently treats blank/whitespace args as 0). How would you like to proceed?

Action:

>

 Fix highest severity issues

Fix all issues

Commit a summary of findings

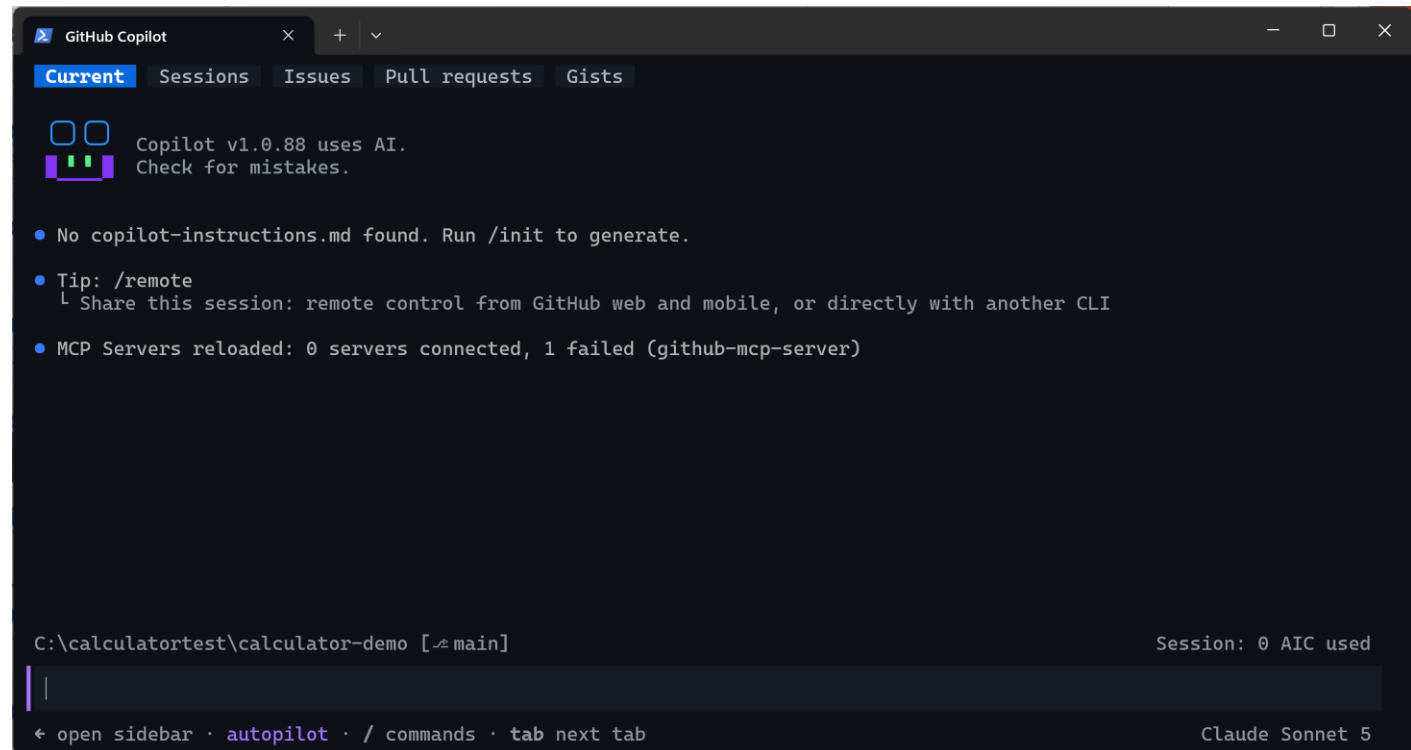
Do nothing for now

Other (type your answer)

↑/↓ select · enter accept · ctrl+d decline · esc cancel

Local delegation

- Copilot can operate autonomously locally or in the cloud
- Autopilot mode
 - Local execution
 - Interactive
 - *Shift-tab* to switch to autopilot mode
- Programmatically
 - Add *--autopilot* flag to *copilot -p*



The screenshot shows the GitHub Copilot application running in a terminal window. The window title is "GitHub Copilot". The interface has a dark theme. At the top, there are tabs: "Current" (selected), "Sessions", "Issues", "Pull requests", and "Gists". Below the tabs, there is a status bar with the text "Copilot v1.0.88 uses AI. Check for mistakes." and a small icon. Below the status bar, there is a list of messages:

- No copilot-instructions.md found. Run /init to generate.
- Tip: /remote
 - ↳ Share this session: remote control from GitHub web and mobile, or directly with another CLI
- MCP Servers reloaded: 0 servers connected, 1 failed (github-mcp-server)

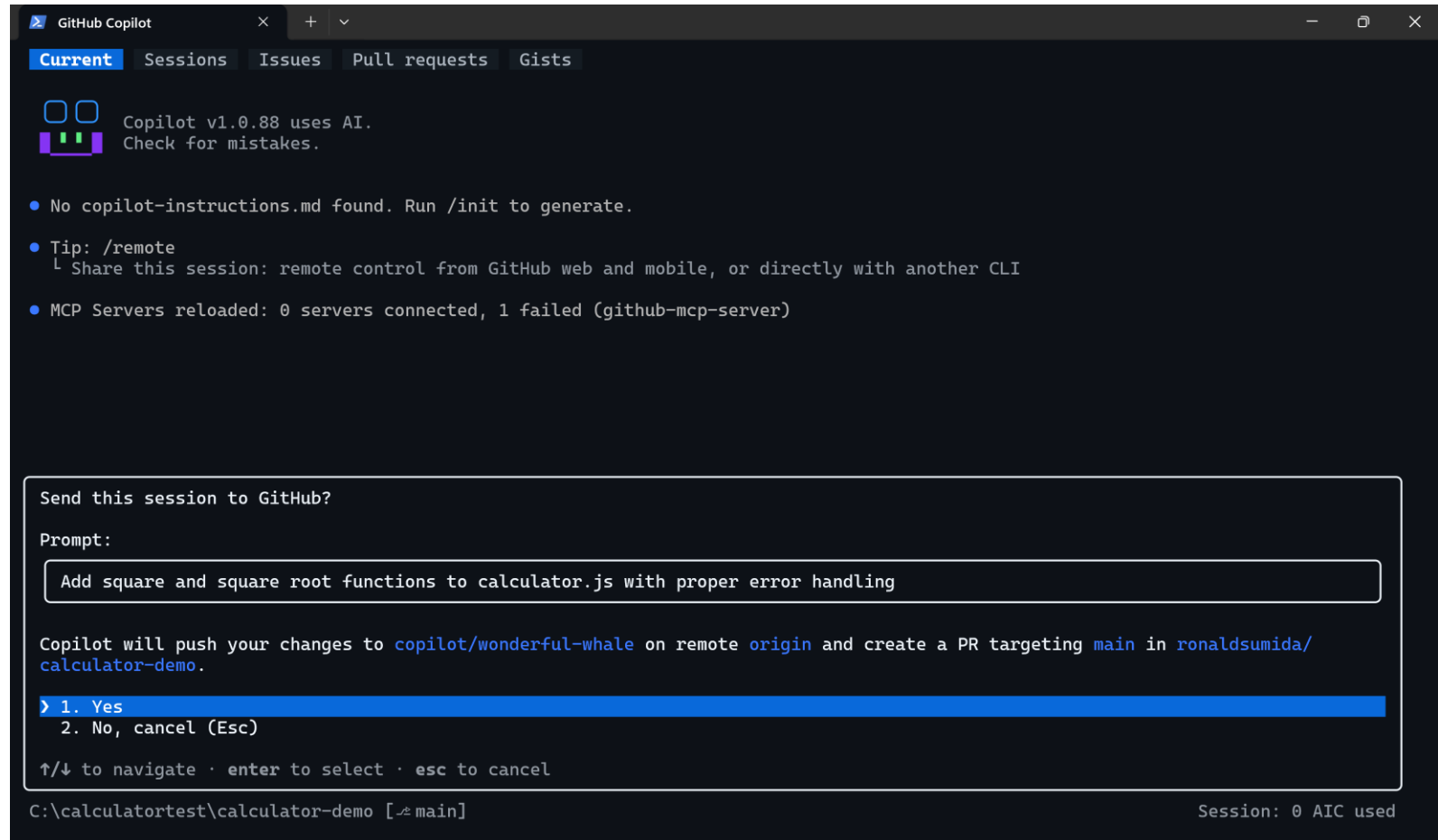
At the bottom of the terminal, there is a prompt showing the current directory: "C:\calculator\test\calculator-demo [main]". To the right of the prompt, it says "Session: 0 AIC used". At the very bottom, there is a footer with the text "← open sidebar · autopilot · / commands · tab next tab" and "Claude Sonnet 5".

Remote delegation

- */delegate* command
 - Runs on GitHub Copilot cloud agent
 - Continues to run even if local machine is shut down
- Steps
 - Create a new branch
 - Draft a pull request
 - Run in the background
 - Request a review

/delegate example

- */delegate Add square and square root functions to calculator.js with proper error handling*



Scheduling prompts

- Experimental feature
 - Need to run with */experimental on*
 - *copilot -i "/experimental"*
- Interactive only
- */every*
 - Prompt is submitted on a recurring basis
- */after*
 - Prompt is submitted once after the specified delay
- Viewing scheduled tasks
 - Run */every* or */after* with no arguments

Scheduling prompts

- */every* [INTERVAL] PROMPT

- Interval

- Time between resubmissions if specified
 - Copilot determines when the prompt is run again
 - Based on the task
 - Value is not fixed
 - Stops when nothing left to watch

- Example:

- */every src/*.js Create comprehensive unit tests for all calculator functions if one doesn't already exist.*

Scheduling prompts

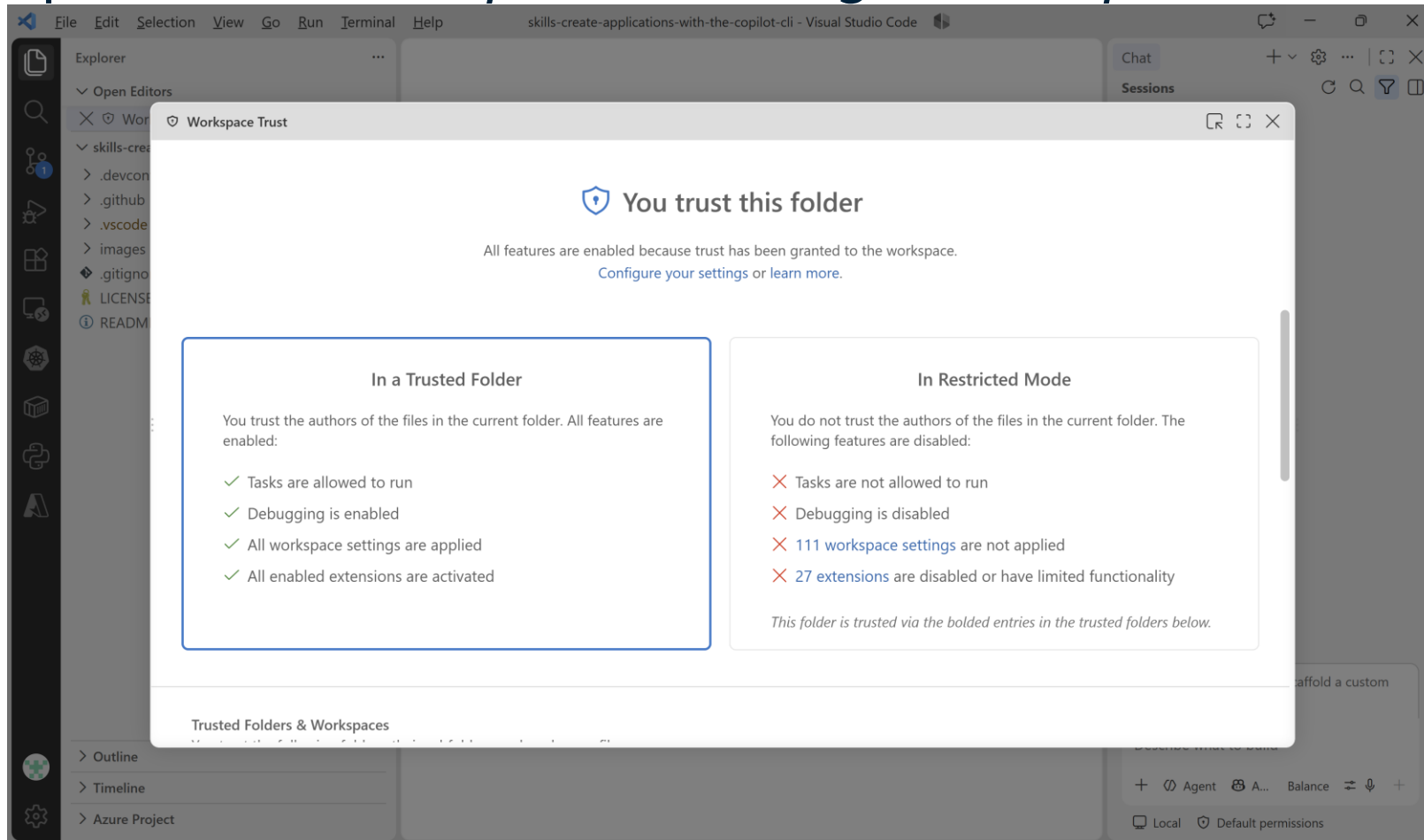
- */after* DELAY PROMPT
 - Removed from the scheduled list after prompt submitted
 - DELAY
 - Must be provided
- Example:
 - */after 11:00 PM run: copilot -p "Summarize all JavaScript files in this repository"> js-summary.md*
 - Runs even if the CLI is closed

Connecting Windows Terminal to VS Code

- Open the same folder in both
- VS Code
 - Make the folder trusted
 - Command palette – *Workspaces: Manage Workspace Trust*
 - Add folder to the list of *Trusted Folders & Workspaces*
- Windows Terminal
 - Connect to the IDE
 - */ide*
 - Select the IDE workspace to connect to

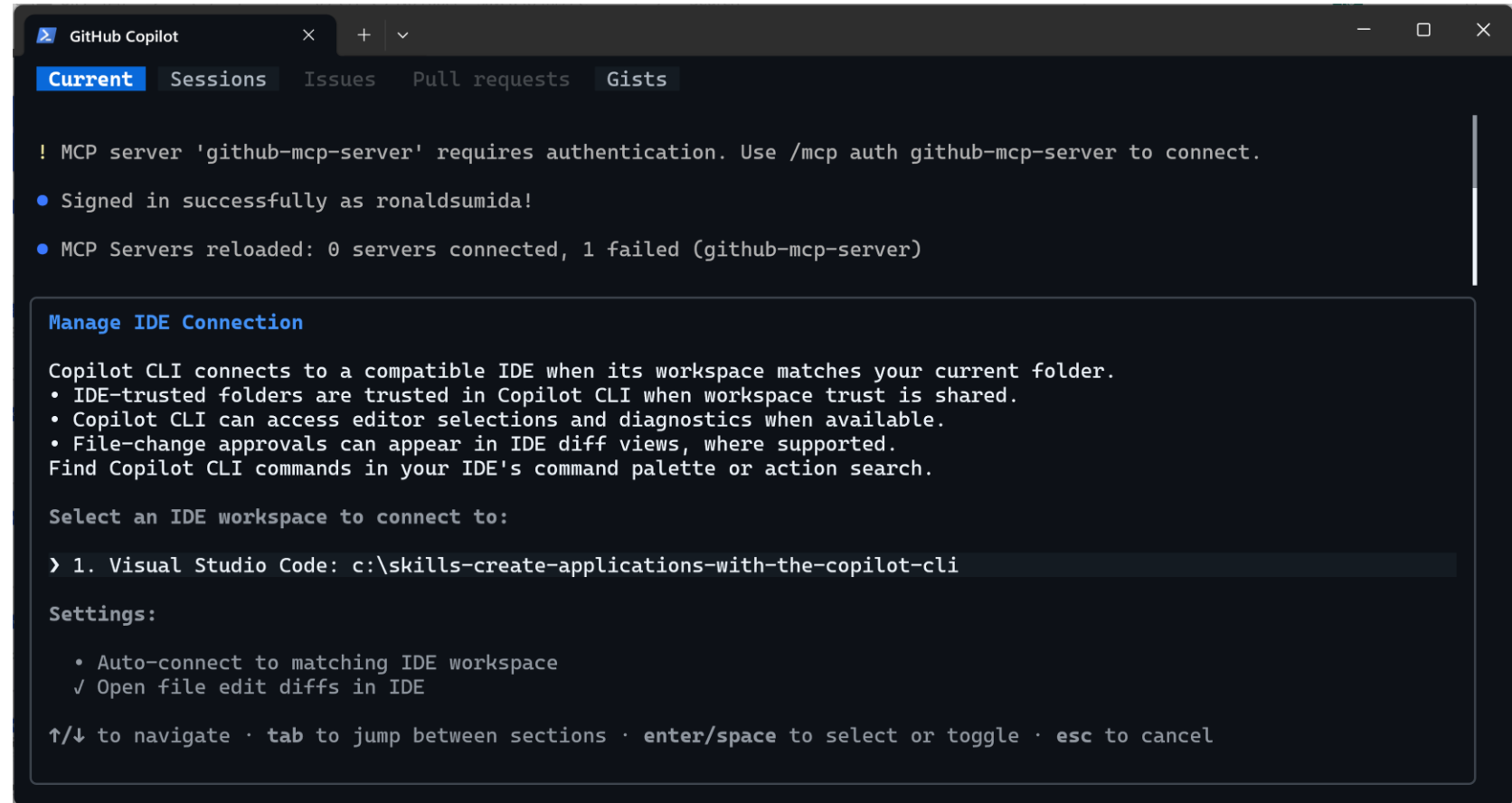
VS Code Workspace Trust

- Command palette – *Workspaces: Manage Workspace Trust*



Windows Terminal – /ide command

- Settings
 - Auto-connect



```
GitHub Copilot
Current Sessions Issues Pull requests Gists

! MCP server 'github-mcp-server' requires authentication. Use /mcp auth github-mcp-server to connect.

• Signed in successfully as ronaldsumida!
• MCP Servers reloaded: 0 servers connected, 1 failed (github-mcp-server)

Manage IDE Connection

Copilot CLI connects to a compatible IDE when its workspace matches your current folder.
• IDE-trusted folders are trusted in Copilot CLI when workspace trust is shared.
• Copilot CLI can access editor selections and diagnostics when available.
• File-change approvals can appear in IDE diff views, where supported.
Find Copilot CLI commands in your IDE's command palette or action search.

Select an IDE workspace to connect to:

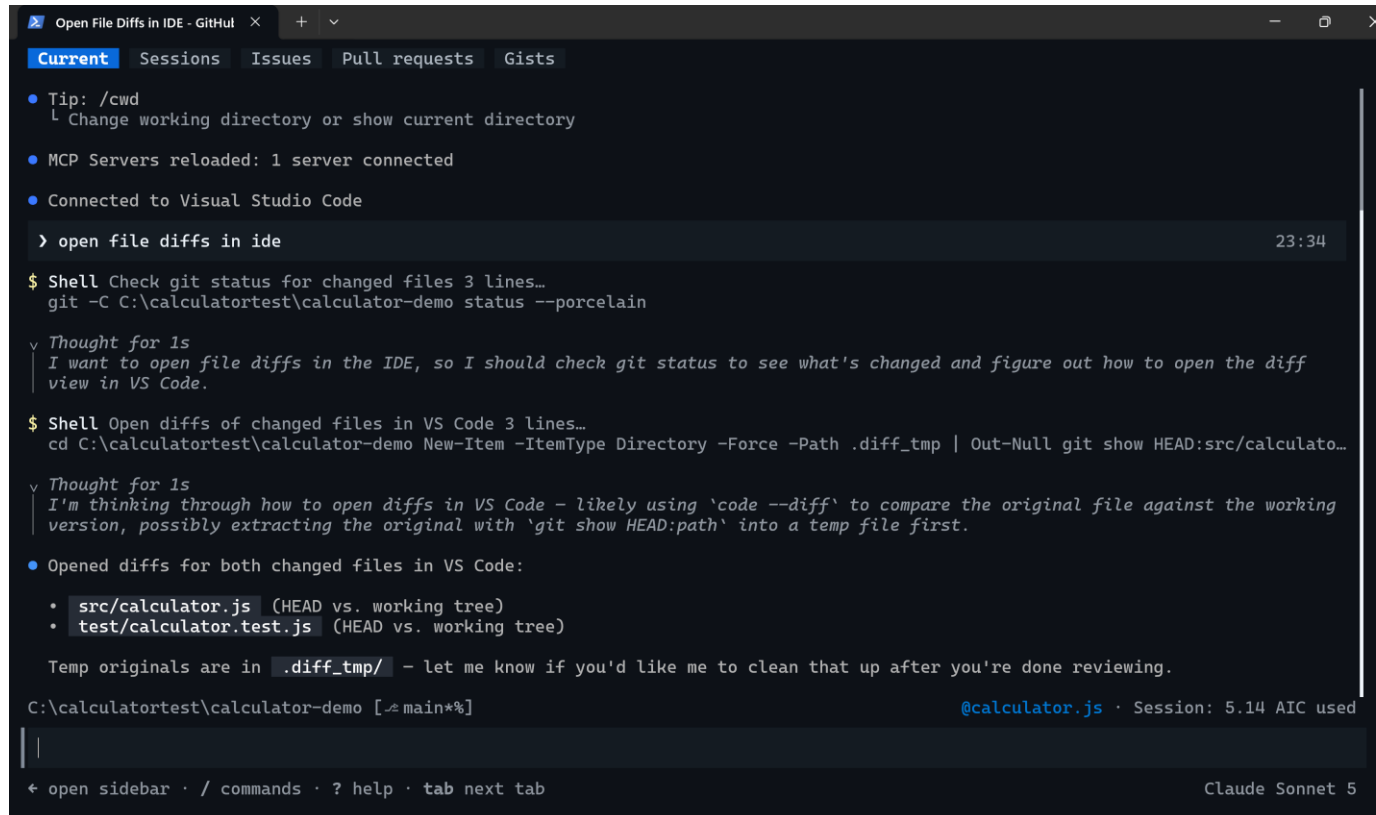
> 1. Visual Studio Code: c:\skills-create-applications-with-the-copilot-cli

Settings:
• Auto-connect to matching IDE workspace
✓ Open file edit diffs in IDE

↑/↓ to navigate · tab to jump between sections · enter/space to select or toggle · esc to cancel
```

Windows Terminal VS Code Integration example

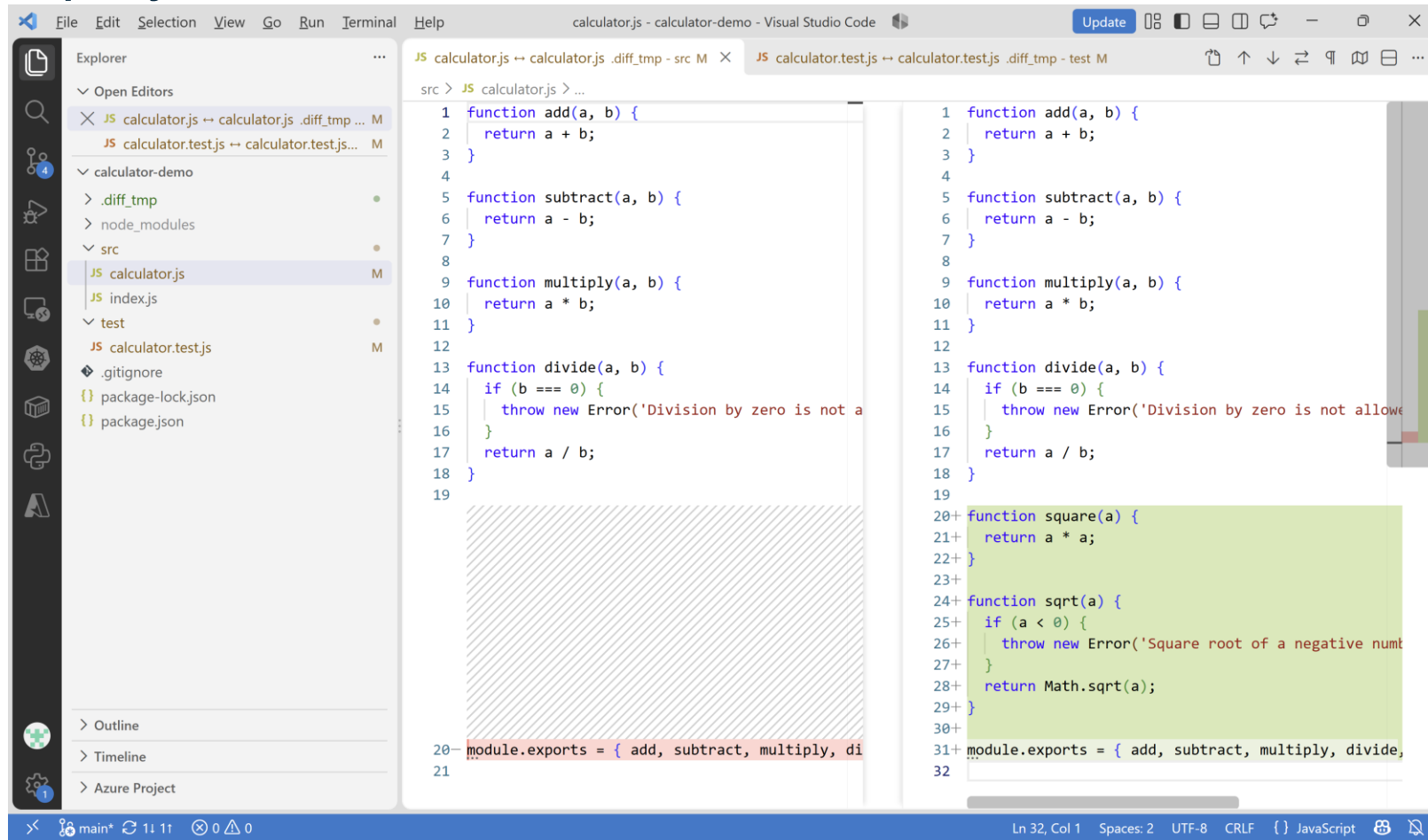
- *Add square and sqrt methods to calculator*
- *Open file diffs in IDE*



```
Open File Diffs in IDE - GitHub
Current Sessions Issues Pull requests Gists
• Tip: /cwd
  ↳ Change working directory or show current directory
• MCP Servers reloaded: 1 server connected
• Connected to Visual Studio Code
> open file diffs in ide 23:34
$ Shell Check git status for changed files 3 lines...
git -C C:\calculatortest\calculator-demo status --porcelain
Thought for 1s
I want to open file diffs in the IDE, so I should check git status to see what's changed and figure out how to open the diff view in VS Code.
$ Shell Open diffs of changed files in VS Code 3 lines...
cd C:\calculatortest\calculator-demo New-Item -ItemType Directory -Force -Path .diff_tmp | Out-Null git show HEAD:src/calculato...
Thought for 1s
I'm thinking through how to open diffs in VS Code - likely using 'code --diff' to compare the original file against the working version, possibly extracting the original with 'git show HEAD:path' into a temp file first.
• Opened diffs for both changed files in VS Code:
  • src/calculator.js (HEAD vs. working tree)
  • test/calculator.test.js (HEAD vs. working tree)
Temp originals are in .diff_tmp/ - let me know if you'd like me to clean that up after you're done reviewing.
C:\calculatortest\calculator-demo [~ main*] @calculator.js · Session: 5.14 AIC used
open sidebar · / commands · ? help · tab next tab Claude Sonnet 5
```

Windows Terminal VS Code Integration example

- Diffs are displayed in VS Code



Lab – GitHub Copilot CLI

- Create applications with the Copilot CLI
 - <https://github.com/skills/create-applications-with-the-copilot-cli>

Summary and conclusions

- GitHub Copilot CLI
 - Reduces context switching
 - Can run headless
 - Can run anywhere
 - Brings AI to terminal-native work
 - Automates repetitive tasks
- Conclusion
 - The CLI has emerged as a popular vehicle for using GitHub Copilot